Computing interactions

Before advancing the particles, though, we need to compute the inter-particle forces. The main trick is to use the fact that all forces have finite radius, and after a certain distance all forces are zero. In my implementation, I only allow forces with radius up to 32. Thus, I know that any pair of particles further than 32 units of distance apart cannot interact at all.

To utilize this, we use the typical approach of spatial hashing/binning: make a square grid with cells/bins of size

$$32 \times 32$$

$$32 \times 32$$

, sort the particles into these bins, then compute interactions only between neighbouring bins. This sounds easy, and is actually almost trivial to implement on the CPU (e.g. in C++ just make a 2D array, each storing `std::vector<particle>`, and you're good to go). However, building such data structures on the GPU takes a lot of effort — we can't, for example, allocate GPU memory directly from a shader!.

Instead, we'll use a linearized structure to store the bins. Here's how it works: we'll have an array of particles, where particles that reside in the same bin occupy a contiguous section of this array. Then, we'll also have an array which stores the start of this section (i.e. the offset) for each bin. We don't need to store the end of it, because the end of a bin is exactly the start of the next bin! We could also store the particle IDs instead of particles themselves, but that's another indirection which could hurt memory access performance later, when we compute the forces.

To do this, we use a typical three-phase approach:

- Phase 1: compute the number of particles in each bin
- Phase 2: run parallel prefix sum to compute the offset for each bin
- Phase 3: place each particle into the corresponding bin

Phases 1 and 3 make heavy use of shader atomics. Phase 1 is fairly simple — run a compute shader for each particle, compute the (linearized) bin index for this particle, and increment the size of this bin:

```
@group(0) @binding(0) var<storage, read> particles : array<Particle>;
```

```
@group(1) @binding(0) var<uniform> simulationOptions : SimulationOptions;

@group(2) @binding(0) var<storage, read_write> binSize : array<atomic<u32>>;

@compute @workgroup_size(64)
fn fillBinSize(@builtin(global_invocation_id) id : vec3u)
{
    if (id.x >= arrayLength(&particles)) {
        return;
    }

    // Read the particle data
    let particle = particles[id.x];

    // Compute the linearized bin index
    let binIndex = getBinInfo(vec2f(particle.x, particle.y),
        simulationOptions).binIndex;

    // Increment the size of the bin
    atomicAdd(&binSize[binIndex + 1], 1u);

}
```

Since this is done in parallel by many shader invocations, we need to use atomics here. I'm actually incrementing array at index `binIndex + 1`, and leaving the 0-th element at the value of 0, because this way `binOffset[i + 1] - binOffset[i]` always gives you the size of the i-th bin. Note that in this case the `binSize` array size is 1 plus the actual number of bins.

Here, `getBinInfo` is just some helper function that computes the linearized bin index from the particle position and the size of the whole grid of bins:

```
fn getBinInfo(position : vec2f, simulationOptions : SimulationOptions) -> BinInfo
{
    let binSize = simulationOptions.binSize;
    let gridSize = simulationOptions.gridSize;

    let binId = vec2i(
        clamp(i32(floor((position.x - simulationOptions.left) / binSize)), 0, gridSize.x - 1),
        clamp(i32(floor((position.y - simulationOptions.bottom) / binSize)), 0, gridSize.y - 1)
    );

    let binIndex = binId.y * gridSize.x + binId.x;
```

```
    return BinInfo(binId, binIndex);

}
```

(N.B.: in my actual code, `gridSize` is not passed as a uniform, but computed in-place, because I did it this way first and then never changed that.)

In Phase 2, we need to convert the array of bin sizes into an array of bin offsets. Since the particles from the same bin go continuously in memory, a bin with some index starts when we've already put all the particles from bins with smaller indices into the array. This means that the offset

$O[i]$

of the i-th bin is simply the sum of the sizes

$S[j]$

of all preceding bins:

$O[i]$

]

=

i

−

1

∑

j

=

0

S

[

j

]

O[i]=∑j=0i−1S[j]

. In pseudocode:

```
offset[0] = 0;
for (int i = 1; i < binCount; ++i)

    binOffset[i] = binOffset[i - 1] + binSize[i];
```

This is known as computing *prefix sums*. This is, of course, not a parallel algorithm. To harness the power of the GPU, we'd want to run a parallel algorithm that does that. This is known, of course, as the *parallel prefix sum* problem, and we'll talk about it in the next section.

For now, let's assume we've computed these offsets somehow. In Phase 3, we need to place ("sort") the particles into a new array, based on which bin this particle belongs to. We do this by running a per-particle compute shader once again. Because we want to put different particles into different indices in the new array, we need to use atomics again:

```
@group(0) @binding(0) var<storage, read> source : array<Particle>;
@group(0) @binding(1) var<storage, read_write> destination : array<Particle>;
@group(0) @binding(2) var<storage, read> binOffset : array<u32>;
@group(0) @binding(3) var<storage, read_write> binSize : array<atomic<u32>>;

@group(1) @binding(0) var<uniform> simulationOptions : SimulationOptions;

@compute @workgroup_size(64)
fn sortParticles(@builtin(global_invocation_id) id : vec3u)
{
    if (id.x >= arrayLength(&source)) {
        return;
    }

    // Read the particle data
    let particle = particles[id.x];

    // Compute the linearized bin index
    let binIndex = getBinInfo(vec2f(particle.x, particle.y),
        simulationOptions).binIndex;

    // Atomically compute the index of this particle
    // within the corresponding bin
    let indexInBin = atomicAdd(&binSize[binIndex], 1);

    // Write the particle into the sorted array
    let newParticleIndex = binOffset[binIndex] + indexInBin;
    destination[newParticleIndex] = particle;


}
```

Here, we need to zero out the `binSize` array before running this shader. Again, since multiple shaders run in parallel, we use atomics to compute the index of a particle within its bin.

Once we've done that, we can easily iterate over particles of a specific bin, which allows us to run the actual shader that computes per-particle forces, by only iterating over the neighbouring bins. Once again, a single shader invocation per each particle:

```
@group(0) @binding(0) var<storage, read_write> particles : array<Particle>;
@group(0) @binding(1) var<storage, read> binOffset : array<u32>;
@group(0) @binding(2) var<storage, read> forces : array<Force>;

@group(1) @binding(0) var<uniform> simulationOptions : SimulationOptions;
```

```
@compute @workgroup_size(64)
fn computeForces(@builtin(global_invocation_id) id : vec3u)
{
    if (id.x >= arrayLength(&particles)) {
        return;
    }

    // Read particle data
    var particle = particles[id.x];
    let type = u32(particle.type);

    // Compute the bin that contains this particle
    let binInfo = getBinInfo(vec2f(particle.x, particle.y), simulationOptions);

    // Compute the range of neighbouring bins for iteration
    var binXMin = binInfo.binId.x - 1;
    var binYMin = binInfo.binId.y - 1;
    var binXMax = binInfo.binId.x + 1;
    var binYMax = binInfo.binId.y + 1;

    // Guard against grid boundaries
    binXMin = max(0, binXMin);
    binYMin = max(0, binYMin);
    binXMax = min(binInfo.gridSize.x - 1, binXMax);
    binYMax = min(binInfo.gridSize.y - 1, binYMax);

    let particlePosition = vec2f(particle.x, particle.y);

    var totalForce = vec2f(0.0, 0.0);

    // Iterate over neighbouring bins
    for (var binX = binXMin; binX <= binXMax; binX += 1) {
        for (var binY = binYMin; binY <= binYMax; binY += 1) {
            // Compute linearized bin index
            let binIndex = binY * binInfo.gridSize.x + binX;

            // Find the range of particles from this bin
            let binStart = binOffset[binIndex];
            let binEnd = binOffset[binIndex + 1];

            // Iterate over particles from this bin
            for (var j = binStart; j < binEnd; j += 1) {
                // Prevent self-interaction
```

```
            if (j == id.x) {
                continue;
            }

            let other = particlesSource[j];
            let otherType = u32(other.type);

            // Apply the inter-particle force

            let force = forces[type * u32(simulationOptions.typeCount) + otherType];

            var r = vec2f(other.x, other.y) - particlePosition;
            let d = length(r);

            if (d > 0.0) {
                let n = r / d;

                totalForce += force.strength * max(0.0, 1.0 - d / force.radius) * n;
                totalForce -= force.collisionStrength * max(0.0, 1.0 - d / force.collisionRadius) * n;
            }
        }
    }
}

    // Update velocity, assuming unit mass
    particle.vx += totalForce.x * simulationOptions.dt;
    particle.vy += totalForce.y * simulationOptions.dt;

    particles[id.x] = particle;


}
```

(My actual code also includes a central force & handling looping boundaries. I also read particles from one buffer and write them to another buffer, but the idea is the same.)

`forces` is a

M

2

M2

-sized array (where

$$M$$

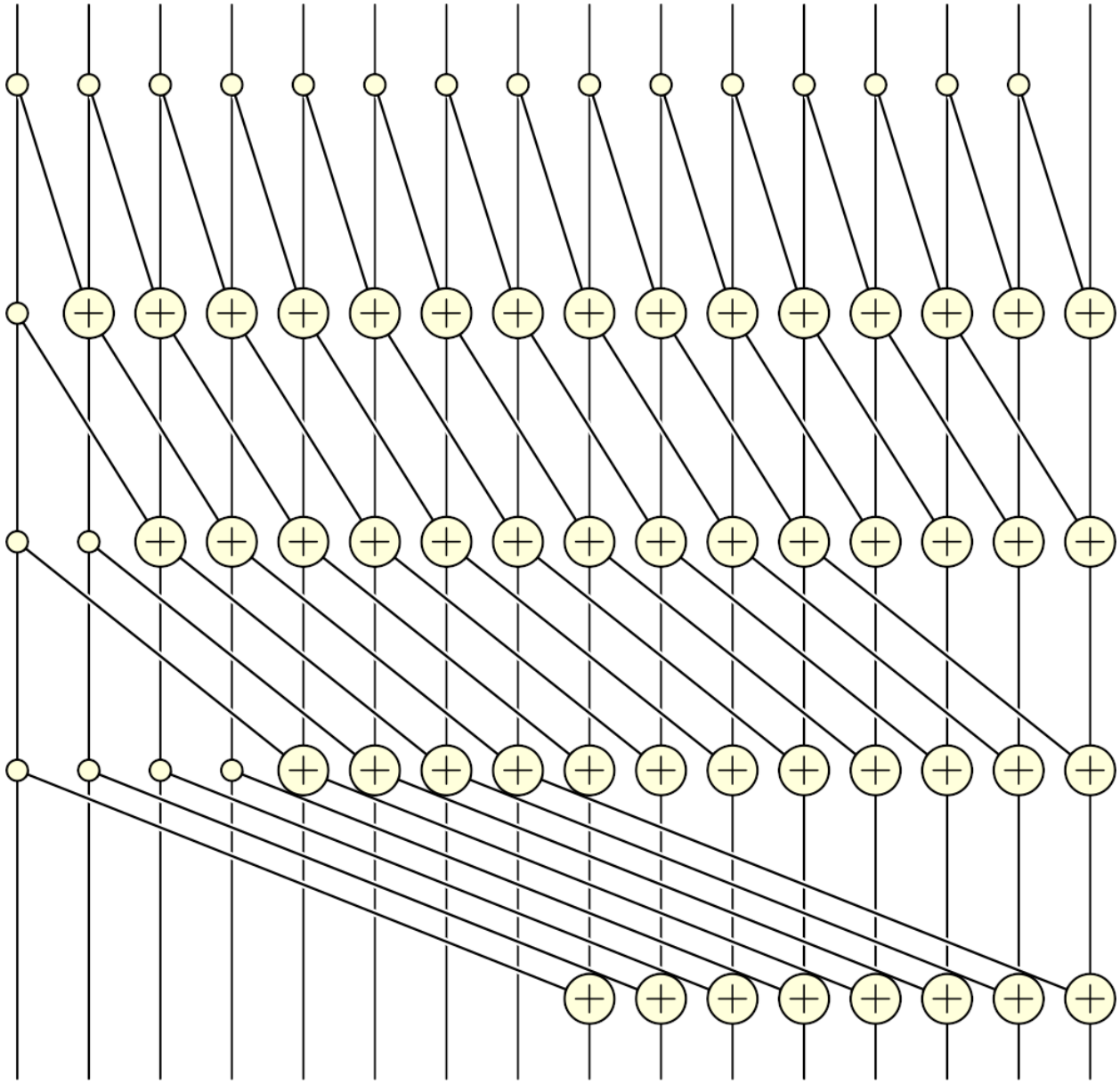
$$M$$

is the number of particle types) which stores the inter-particle force parameters.

Parallel prefix sum

Computing prefix sums in parallel is a well-known problem. I'm using the simplest solution, outlined as the *Algorithm 1* in the wikipedia article, and also called the *naive parallel scan* in this GPU Gems 3 article.

The image from wikipedia explains everything rather nicely:



The main idea is to run several sweeps over the whole array

$$\log_2(N)$$

log2(N)

times (where

N

N

is the array size), each time adding `x[i-step]` to `x[i]`, where `step` is a power of 2. In pseudocode, at iteration `k` we do

```
step = (1 << k);
parallel for (int i = step; i < array.size; ++i)
    array[i] += array[i - step];
```

We start iteration from `i = step` because otherwise at `array[i - step]` we'd try reading at an address before the array starts. This code won't actually work because overwriting `array[i]` affects what other invocations (with different values of `i`) will read, so we have to use 2 arrays: one for reading, and one for writing. Something like this:

```
parallel for (int i = 0; i < input.size; ++i) {
    if (i < step)
        output[i] = input[i];
    else
        output[i] = input[i] + input[i - step];

}
```

Then, to compute the actual prefix sum, we create a temporary array, and we ping-pong the input & temporary arrays

log

2

(

N

)

log2(N)

times with `step` taking the values of increasing powers of 2. On the first iteration (`k = 0, step = 1`), our `binSize` array is the input, and the temporary array is the output. On the next iteration (`k = 1, step = 2`), the temporary array is the input, and the `binSize` array is the output, and so on. In terms of WebGPU, this requires creating two bind groups with the same layout, one that reads from `binSize` array and writes to temporary array, and the other does the opposite.

Now, with this method, the resulting prefix sum will be in the first or the second array, depending on the parity of the number of steps performed, i.e. the parity of

log

2

(

N

)

log2(N)

. That's rather inconvenient, so I always round it up to the closest even number: this way, the result is always in the same `binSize` array, while an extra prefix sum iteration would simply copy one buffer into the other automatically (look what happens with the code when `step >= array.size`).

That's a lot of talking, but the implementation is rather simple:

```
@group(0) @binding(0) var<storage, read> input : array<u32>;
@group(0) @binding(1) var<storage, read_write> output : array<u32>;
@group(0) @binding(2) var<uniform> stepSize : u32;

@compute @workgroup_size(64)
fn prefixSumStep(@builtin(global_invocation_id) id : vec3u)
{
    if (id.x >= arrayLength(&input)) {
        return;
    }
```

```
    if (id.x < stepSize) {
        output[id.x] = input[id.x];
    } else {
        output[id.x] = input[id.x - stepSize] + input[id.x];
    }

}
```

In order to supply different `stepSize` values while still keeping the whole thing as a single compute pass, I have a buffer with all `stepSize` values I need, and use it as the source for the uniform `stepSize` value with proper dynamic offsets in the `GPUComputePassEncoder.setBindGroup` call.

So, with

log

2

(

N

)

log2(N)

iterations (rounded up to an even number), we run this compute shader (one invocation per bin, — or, in my case, `binCount + 1` invocations for reasons I outlined earlier), while ping-ponging two buffers. Here's what it looks like in JavaScript:

```
prefixSumIterations = Math.ceil(Math.ceil(Math.log2(binCount + 1)) / 2) * 2;

...

binningComputePass.setPipeline(binPrefixSumPipeline);
for (var i = 0; i < prefixSumIterations; ++i) {
    binningComputePass.setBindGroup(0, binPrefixSumBindGroup[i % 2], [i * 256]);
    binningComputePass.dispatchWorkgroups(Math.ceil((binCount + 1) / 64));

}
```

(The `i * 256` dynamic offset is due to requirements that dynamic offsets must be a multiple of 256 bytes. The `Math.ceil((binCount + 1) / 64)` workgroup count is because the shader uses a workgroup size of 64, and I run it on an array of size `binCount + 1`.)

I could use a more sophisticated prefix sum algorithm, but there was no need to: even with e.g. 10000 bins, this whole particle sorting & prefix sum computation takes about 0.1ms on my GeForce GTX 1060. The whole performance is *always* dominated by inter-particle forces computation, so optimizing the binning step gives little profit.